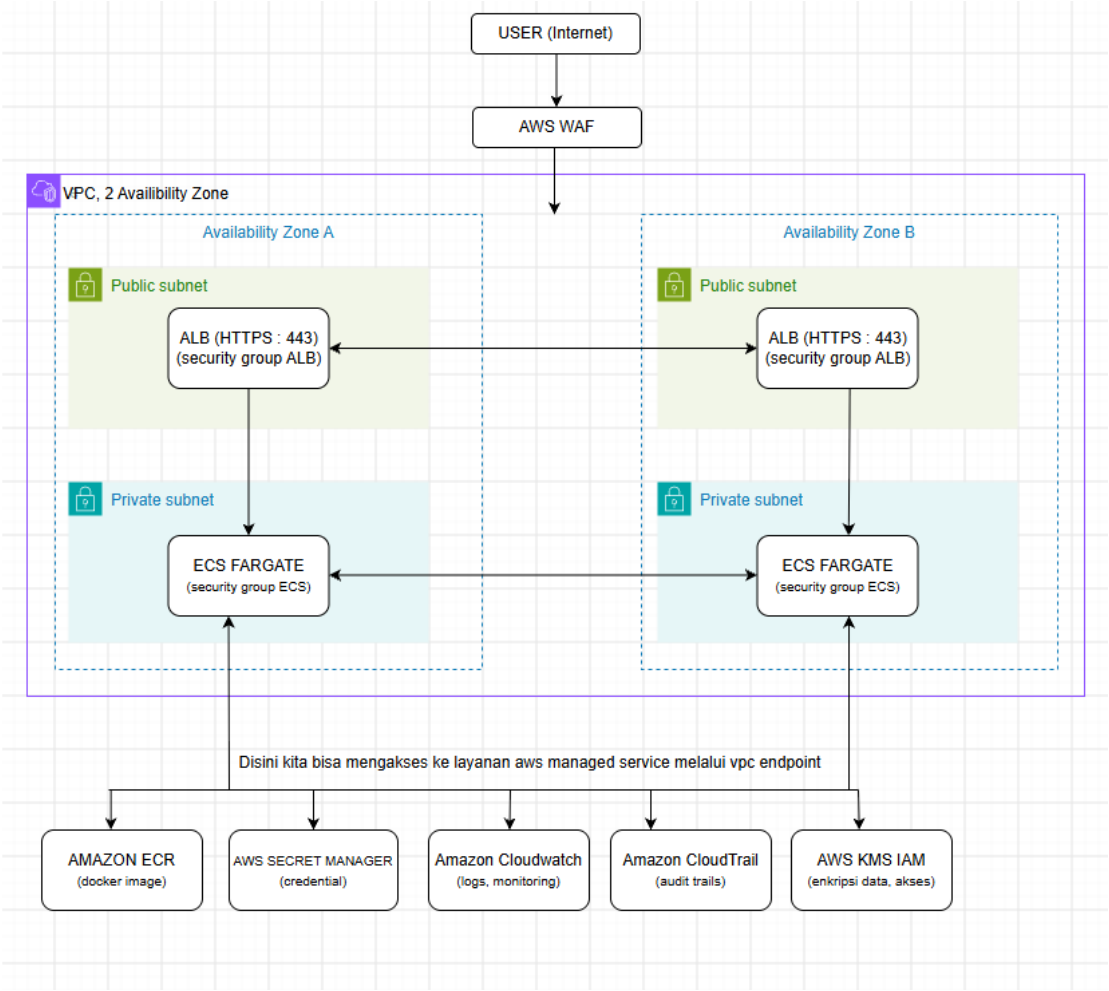
DevSecOps Engineer Home Test

Name : Andi Manjulang Lumban Gaol

Date : 27 September 2026

a. Task 1 : AWS Architecture and Security Design



Requirement :

1. Explain the request flow from the internet to the application?
   Jawaban :
   - User mengakses domain backend API misalnya **https://api.devsecopsindico.com** dari internet.

   - Lalu request akan melewati **AWS WAF(Web Application Firewall)** yang dimana AWS WAF ini berfungsi untuk menyaring request yang berpotensi mencurigakan seperti sql injection dan jika request tidak diizinkan maka dari WAF akan menghentikannya.

- Dan jika request diizinkan/berhasil maka request akan masuk ke **ALB(Load Balancer)** yang ada di public subnet dan hanya mengizinkan koneksi 443/HTTPS di security group ALB nya.

  - sg-alb(public subnet)

  inbound : 443 dari 0.0.0.0/0
  outbound : ke sg-ecs (port aplikasi)

  - sg-ecs (private subnet)
  inbound : si port aplikasi (misalkan : 8080) hanya dari sg-alb
  outbound : ke layanan AWS (via vpc endpoint)

- Kemudian ALB akan meneruskan request ke container aplikasi kita yang ada di **AWS ECS FARGATE** (layanan aws untuk menjalankan aplikasi berbasis container tanpa harus mengelola server ec2 kita sendiri) melalui **TARGET GROUP** yang ada di private subnet.

2. Identity which resources should be public and which should be private?
   Jawaban :
   - ALB(Load Balancer) dibuat di public subnet karena sebagai pintu masuk si aplikasi kita dari internet, jadi si user bisa mengakses aplikasi melalui HTTPS dengan port 443

   - ECS Fargate harus dibuat di private subnet karena agar container backend kita tidak dapat diakses langsung dari internet dengan private subnet dan tanpa public ip, sehingga request yang masuk itu hanya diterima oleh ALB di security group agar aman dari serangan dan dapat dikontrol.

3. Explain how the application retrieves secrets without hardcoding them?
   Jawaban :
   - Dari sisi keamanan, kita itu tidak boleh menulis/hardcode credential kita secara langsung dalam source code kita, jadi secret kita itu disimpan di AWS SECRET MANAGER.

   - Kita bisa menggunakan ECS Task Definition, dan berikan akses secretmanager:GetSecretValue kepada ECS Task Excecution Role, di task definition kita tulis ARN dan bukan credential aslinya seperti :

     ```
     “secret”: [
       {
         “name”: “DB_PASS”,
         “valueFrom”: “arn:aws:secretmanager:ap-southeast-1:00112233:secret:prod/backend-
           api/db-credential/”
       }
     ]
     ```

   - Kemudian ECS akan mengambil credentialnya saat task start dengan menggunakan Task Execution Role dan memasukkan nya ke container sebagai environment variable

4. Describe the IAM roles requires by the CI/CD pipeline and the running application?
   Jawaban :
   Kita jangan menggunakan satu IAM Role untuk seluruh kebutuhan CI/CD kita, jadi kita harus memisahkan role berdasarkan tugasnya misalkan menggunakan Gitlab CI seperti :
   - CI build role, yang bertugas di pipeline pada tahap build dan contoh izin nya untuk push docker image kita ke satu repo ECR kita ataupun ke repo tertentu sesuai kebutuhan

   - CI deploy role, yang bertugas di pipeline pada tahap deploy dan contoh izin nya untuk mengupdate ECS service tertentu kita sesuai kebutuhan

   - ECS Task Execution Role, yang bertugas untuk menjalankan kebutuhan awal dari task dan contoh izin nya untuk mengambil secret/credential

5. Explain how you would prevent unauthorized acces to production resources?
   Jawaban :
   - Saya akan selalu memastikan kepada user yang menggunakan resource production misal dari AWS untuk membuat MFA(Multi Factor Authentication) agar dapat diverifikasi melalui 2 faktor melalui username dan password dan memasukkan kode dari aplikasi autentikator yang sudah dihubungkan dengan AWS production

   - Saya akan memisahkan akun di environment Staging, Development, dan terutama Production serta akan membatasinya menggunakan IAM Roles, misal membuat role khusus seperti prod-deployment jadi hanya user yg terdaftar di role tersebut yang bisa akses untuk environment production

   - Saya juga akan memastikan dengan adanya manual approval agar dapat mengecek dan menganalisis perubahan sebelum terjadinya akses ke production

6. Identity at least five security risks and corresponding mitigation?
   Jawaban :

   - Jika secret/credential kita ter commit ke source code ke Gitlab, misal AWS_ACCESS_KEY_ID dan AWS_SECRET_KEY_ID , maka kita bisa menonaktifkan credential acces key dan secret key tersebut dan membuat credential yang baru di AWS

   - Kita juga harus membatasi IAM Permission, yang dimana hanya user tertentu/atasan yang memiliki akses misalnya administrator-acces untuk menjaga akses keamanan dari environment kita

   - Security group yang membuka port 22/SSH ataupun database kita terbuka ke 0.0.0.0/0 yang dimana bisa diakses langsung dari internet, maka dari itu kita harus batasi source ip sesuai dengan kebutuhan dan selalu menggunakan database ke private subnet agar tidak bisa dapat diakses langsung dari internet

- Jika docker image memiliki CVE(Common Vulnerabilities and Exposures) yaitu ID untuk mengidentifikasi suatu kerentanan tertentu, kita bisa menggunakan tools Trivy untuk scan adanya kerentanan depedensi dari image tersebut

- Jika di pipeline CI/CD tidak memiliki security checks, maka pipeline kita akan berisi secret/credential yang akan bisa lolos, maka dari itu kita harus menambahkan stage secret scanning ataupun dependency scanning untuk ngescan adanya secret leaks pada pipeline kita jadi jika ditemukan secret leaks pada pipeline kita itu akan otomatis stop dan tidak lanjut ke tahapan deployment

b. Task 2 : CI/CD Security Pipeline

Requirement :

Create CI/CD pipeline for a sample backend application using Gitlab CI and the pipeline must demonstrate the following stages:

1. Contoh pipeline **“test.gitlab-ci.yml”**

```
stages:
  - test
  - security
  - build
  - scan
  - dast
  - deploy

# Source Validation
source_validation:
  stage: test
  image: node:22
  script:
    - npm ci
    - npm test

# SAST - SonarQube
sast_scan:
  stage: security
  image:
    name: sonarsource/sonar-scanner-cli:latest
    entrypoint: [""]
  script:
    - sonar-scanner -Dsonar.projectKey=backend-api -Dsonar.sources=. -Dsonar.qualitygate.wait=true

# SCA - Trivy
sca_scan:
  stage: security
  image:
    name: aquasec/trivy:latest
    entrypoint: [""]
  script:
    - trivy fs . --scanners vuln –format json --output sca-report.json
    - trivy fs . --scanners vuln --severity CRITICAL --exit-code 1
# Secret Scanning - Gitleaks
secret_scan:
  stage: security
  image:
    name: ghcr.io/gitleaks/gitleaks:latest
    entrypoint: [""]
```

```
  script:
    - gitleaks git . --redact --exit-code 1

# IaC Scanning - Trivy
iac_scan:
  stage: security
  image:
    name: aquasec/trivy:latest
    entrypoint: [""]
  script:
    - trivy config ./terraform --format json --output iac-report.json
    - trivy config ./terraform --severity CRITICAL --exit-code 1
# Docker Build
docker_build:
  stage: build
  image: docker:27
  services:
    - docker:27-dind
  variables:
    DOCKER_HOST: tcp://docker:2375
    DOCKER_TLS_CERTDIR: ""
  script:
    - docker build -t backend-api:$CI_COMMIT_SHA .
    - docker save backend-api:$CI_COMMIT_SHA -o image.tar
  artifacts:
    paths:
      - image.tar

# Container Scanning - Trivy
container_scan:
  stage: scan
  image:
    name: aquasec/trivy:latest
    entrypoint: [""]
  script:
    - trivy image --input image.tar --format json --output container-report.jsonm
    - trivy image --input image.tar --severity CRITICAL --exit-code 1
# DAST - OWASP ZAP
dast_scan:
  stage: dast
  image: ghcr.io/zaproxy/zaproxy:stable
  script:
    - zap-baseline.py -t "$DAST_TARGET_URL" -I

# Deployment Simulation
deploy:
  stage: deploy
  image: alpine:latest
  script:
```

```
    - echo "Task Security Berhasil. Lanjut ke Deploy!"
  when: manual
```

- Security scanning run automatically on code changes?
  Jawaban :
  Pada pipeline, (sast_scan, sca_scan, secret_scan, iac_scan, container_scan, dast_scan) yang dimana Gitlab menjalankan job jika pipeline ketrigger, misalnya ketika ada push atau commit

- Pipeline artifacts or logs provide evidence of scan result?
  Jawaban:
  Pada pipeline, (artifacts:, path:) untuk menyimpan hasil sebagai artifact

- Critical findings trigger a failed pipeline?
  Jawaban:
  Pada pipeline, (--severity CRITICAL --exit-code 1 pada job trivy) Jika trivy menemukan CRITICAL maka job akan gagal dan tidak sampai ke job deploy

- Lower-severity findings are handled according to a documentacy policy?
  Jawaban:
  Pada pipeline, (trivy fs . --scanners vuln –format json --output sca-report.json) untuk pipeline contoh sekarang dikonfigurasi menjadi CRITICAL dan menggunakan --exit-code 1 untuk menggagalkan job jika ada temuan CRITICAL, dan jika ada temuam HIGH,MEDIUM,LOW akan tetap tercatatat sesuai prioritasnya

- Production deployment requires an apporiate approval or protected deployment mechanism?
  Jawaban:
  Pada pipeline, (when: manual) pada job deploy, deploymeny dijalankan secara manual untuk production agar hanya user berwewenang menjalankan proses deployment

- Secret must not be printed in logs or embedded in the repository?
  Jawaban :
  Pada pipeline, (gitleaks git . --redact --exit-code 1) maka si gitleaks akan mendeteksi di repository dan --redact untuk menyamarkan secret yang misalkan ada pada output gitleaks

- Pipeline job should use least-privilege credential?
  Jawaban:
  Setiap job yang ada di pipeline Gitlab CI/CD harusnya akan diberikan akses yang sesuai untuk menjalankan tugasnya, jadi credential itu disimpan di Gitlab CI/CD variable dan tidak ditulis langsung didalam repository

- Untuk kesimpulan security gates, jadi setiap job dari contoh pipeline diatas adalah untuk menentukan apakah pipeline bisa dilanjut ke tahap berikutnya jika hasil dari masing masing security itu sukses maka akan dilanjut sampai ke tahapan deployment

- Untuk kesimpulan False Positives, adalah kondisi ketika tools security scanning mendeteksi suatu kerentanan, tetapi setelah diperiksa ternyata bukan vulnerability yang benar benar berbahaya contohnya : ketika Gitleaks mendeteksi sebuah file didalam repository kita sebagai secret/credential, tapi setelah diperiksa, script tersebut hanya contoh dari API KEY palsu yang digunakan sebagai dokumentasi dan bukan credential yang asli.

  Jadi penanganannya, dari tim kita akan memeriksa Kembali script tersebut yang terdeteksi oleh Gitleaks, kemudian memastikan Kembali bahwa script tersebut hanya contoh dari API KEY dan bukan credential yang asli

- Untuk kesimpulan Accepted Risks, adalah kondisi ketika suatu resiko kerentanan memang ada, tetapi diterima sementara karna alasan tertentu, misalkan belum bisa di fix di saat itu juga
  contohnya : ketika Trivy mendeteksi adanya vulnerability di kategori HIGH nih pada salah satu depedensi aplikasi kita, tetapi versi dari depedensi itu belum bisa digunakan karna beresiko membuat aplikasi mengalami error, jadi tim memutuskan untuk tetap menggunakan versi yang saat ini dan untuk sementara waktu untuk menyiapkan perbaikan.

  Jadi penanganannya, dari tim kita akan mencatat vulnerability yang ditemukan oleh Trivy, kemudian menjelaskan alasan dari vulnerability belum bisa di fix dan impactnya, serta akan menentukan batas waktu untuk mengevaluasi dan memperbaiki vulnerability tersebut.

c. Task 3 : Terraform and AWS IAM SECURITY

Requirement :

Create terraform configuration for a minimal AWS Deployment, or improve a provided example if you prefer to use your own repository

TERRAFORM, adalah tools Infrastructure as Code(IaC) yang digunakan untuk membuat dan mengelola suatu infrastructure yang bersifat deklaratif (kita mendefenisikan hasil yang kita mau)

1. Contoh Folder terraform, “test-terraform” yang berisi :
   “main.tf”, sebagai tempat utama kita untuk mendefinisikan resource infrastructure kita
   “variable.tf”, untuk mendefiniskan variable konfigurasi kita agar dapat digunakan kembali
   “.gitignore”, bukan file bagian dari terraform tetapi file konfigurasi untuk Git yang berfungsi mencegah file sensitive ter-commit ke repository kita

   main.tf

```
provider "aws" {
  region = "ap-southeast-1"
}

resource "aws_s3_bucket" "test_bucket" {
  bucket = "test- terraform-bucket"
}
```

   Keterangan :

- Provider “aws”, kita menentukan bahwa terraform akan menggunakan cloud provider AWS
- Region “ap-southeast-1”, kita menentukan region AWS, yaitu Singapore
- Resource “aws_s3_bucket”, perintah untuk terafform membuat S3 bucket
- Bucket "test-terraform-bucket", Nama dari bucket kita dan harus unik karna bersifat global

   variable.tf

```
variable “aws_region” {
  description       = “AWS REGION”
  type              = string
  default           = “ap-southeast-1”
}

variable “bucket_name” {
  description       = “nama s3 bucket”
  type              = string
  default           = “test-terraform-bucket”
}
```

   Keterangan :

- variable “bucket_name”, untuk menentukan nama S3 bucket yang akan dibuat terraform
- description “nama S3 bucket”, memberikan keterangan bahwa variable ini digunakan untuk menyimpan nama S3 bucket

- type “string”, menentukan bahwa variable itu berupa teks atau string dan default “test-terraform-bucket”, menentukan nama default S3 bucket

.gitignore

```
# Terraform folder
.terraform/

# Terraform state
*.tfstate

# Terraform state
*.tfvars

# Terraform crash logs
crash.log
```

Keterangan :

- .terraform/ berfungsi untuk mengabaikan folder yang dibuat otomatis ketika menjalankan “terraform init”
- *.tfstate berfungsi untuk mencegah file terraform state masuk ke Git yang berpotensi berisi data sensitive
- *.tfvars berfungsi untuk mencegah variable “.tfvars” masuk ke Git yang berpotensi berisi credential/password
- crash.log berfungsi untuk mengabaikan file log yang dihasilkan ketika terafform mengalami crash

2. Scenario A : Overly permissive IAM
   A developer proposes the following policy for an application deployment role :

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Action": “*”,
   "Resource": “*”
  }
 ]
}
```

Jawaban:
Dari policy tersebut kita melihat :

- Effect : “Allow”, menentukan bahwa tindakan yang ada dalam policy kita itu diizinkan
- Action: “*”, dapat memberikan izin untuk melakukan semua tindakan pada AWS
- Resource: “*”, dapat memberikan izin terhadap seluruh resource AWS

Maka ini policy ini beresiko berbahaya karna dapat memberikan akses yang sangat bebas, apabila credential kita itu disalahgunakan, si penyerang bisa berpotensi untuk mengakses, bahwa menghapus resource yang ada dalam AWS.

Solusinya :

- Kita menggunakan IAM Least privilege (Prinsip yang memberikan hak akses/permission kepada user/service sesuai dengan kebutuhan tugasnya)
  dengan contoh :
  JSON, Policy ini hanya mengizinkan untuk baca object di bucket yang ditentukan

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Action": [
    "s3:GetObject"
   ],
   "Resource": "arn:aws:s3:::test-app-bucket/*"
  }
 ]
}
```

Keterangan :

- Version, menentukan versi standar dari kebijakan IAM yang digunakan oleh AWS
- Statement, berisi aturan permission yang diberikan kepada user atau role IAM
- Effect: “Allow”, menentukan bahwa tindakan yang ada dalam policy kita itu diizinkan
- Action: “s3:GetObject”, memberikan izin hanya untuk membaca dari S3 bucket
- Resource, menentukan resource dari AWS yang boleh diakses yaitu object di dalam S3 bucket “test-app-bucket”
- */, berarti bahwa izin membaca itu berlaku untuk seluruh object/file yang ada dalam bucket tersebut

Scenario B : Publicy accessible resource
A Terraform change creates an application databases that is accessible from 0.0.0.0/0 and explain how your pipeline detect and block this configuration before deployment?
Jawaban :

- Dari perubahan Terraform tersebut maka database kita akan dapat diakses secara langsung dikarenakan 0.0.0.0/0 yang berarti akses diizinkan dari semua alamat IP

- Sebelumnya di pipeline kita itu harus menggunakan Trivy untuk mendeteksi adanya kerentanan terhadap konfigurasi terraform seperti contoh: trivy config . --severity CRITICAL --exit- code 1, maka jika Trivy mendeteksi adanya kerentanan seperti akses database dan temuan severity status CRITICAL, job scanning akan gagal dan proses pipeline berhenti dan tidak akan lanjut sampai proses deployment

Scenario C: Secret Exposure
An application password is stored in terraform variables and appears in a state file, Explain how you would prevent further exposure, secure the secret, and handle the exisiting exposed credential and terraform state?
Jawaban :
Jika password dari si aplikasi kita itu disimpan langsung di variable terraform dan muncul di terraform state(.tfstate), maka password tersebut berpotennsi bisa terbaca oleh orang lain yang memiliki akses ke file tersebut, solusinya adalah :

- Password kita tidak boleh di harcode langsung di source code/repository, kita bisa simpan password lebih aman di AWS SECRET MANAGER
- Membatasi akses terraform state melalui IAM dan hanya user yang memiliki kewenangan yang bisa mengakses file tersebut
- Menggunakan file .gitignore agar mencegah file sensitive ter-commit

d. Task 4 : Container and Application security

Requirement :

Create a Dockerfile for a small backend application or use an existing sample application and demonstrate the following :

1. Contoh Folder small backend, “test-backend-app” yang berisi:
   “app.js”
   “package.json”
   Dockerfile

   app.js

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello from Backend API!");
});

server.listen(3000, "0.0.0.0");
```

   package.json

```
{
  "name": "test-backend-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js"
  }
}
```

Dockerfile

```
FROM node:22-alpine

WORKDIR /app

COPY app.js package.json ./

USER node

EXPOSE 3000

CMD ["npm", "start"]
```

Keterangan :

- FROM, menggunakan base image Node.js versi 22-alpine
- WORKDIR, menentukan direktori di dalam container
- COPY, menyalin file aplikasi ke container
- USER node, menjalankan aplikasi sebagai non-root
- EXPOSE, membuka port yang digunakan oleh aplikasi
- CMD, perintah utama yang dijalankan saat container berjalan

2. Build image menggunakan command : docker build -t test-backend-app .
3. Setelah berhasil membuat image maka kita lanjut untuk mendeteksi image yang kita buat tadi menggunakan Trivy dengan command :
   trivy image test-backend-app, jadi hasil vulnerability itu bisa kita ketahui setelah kita menjalankan scan menggunakan trivy.
   Kita juga harus menentukan bagaimana vulnerability itu ditangani berdasarkan tingkat kerentanannya seperti :
   - CRITICAL : Memblokir release secara default dan tim akan melakukan perbaikan
   - HIGH : Memprioritaskan perbaikan dan blokir release sesuai tingkat resiko
   - MEDIUM : Mencatat/dokumentasi vulnerability dan jadwal perbaikan kepada tim
   - LOW : Memantau dan melakukan perbaikan sesuai tingkat resiko
4. Jalankan container menggunakan command : docker run -d -p 3000 –name backend test-backend-app
5. Kemudian buka http://localhost:3000 di browser dan akan terlihat respon nya.

e. Task 5 : Troubleshooting and incident response

Scenario A : Pipeline security failure

A pipeline fails because Trivy reports a critical vulnerability in a base image.The development team says the application is working correctly and requests an immediate production deployment.

What would you do? Your response should address risk assessment, remediation, exceptions, and release authorization.

Jawaban:

- Pertama, saya akan melakukan investigasi dengan memeriksa hasil scan dari Trivy untuk mengetahui base image yang memiliki vulnerability CRITICAL dari base image tersebut, kemudian jika terdapat detail vulnerability nya seperti adanya CVE dan apakah ada dampak yang akan mempengaruhi si aplikasi.
- Kedua, saya mengecek penyebab nya apakah base image yang digunakan itu memiliki vulnerability yang belum diperbaiki ataupun menggunakan base image dengan dependensi yang sudah lama
- Ketiga, saya akan memperbaharui base image ke versi terbaru jika base image tersebut sudah tersedia dan ngebuild serta menjalankan trivy scan lagi pada base image yang sudah diperbaiki
- Terakhir walaupun si aplikasi kita dari info tim development itu berjalan dengan baik, jika ada vulnerability dengan status CRITICAL tetap harus ditangani karna akan dapat berpotensi menimbulkan kerentanan pada aplikasi kita kedepannya.

Scenario B : ECS Application cannot retrieve secrets

An ECS Fargate task fails to start because it cannot retrieve a secret from AWS Secrets Manager Identify the AWS components and permissions you would investigate

Jawaban :

Jika aplikasi di ECS Fargate gagal mengambil secret/credential dari AWS SECRET MANAGER, sehingga container kita itu tidak berjalan

- Dikarenakan penyebab nya sudah diketahui yaitu dari ECS Fargate gagal mengambil secre/credential dari AWS SECRET MANAGER, maka kita harus memeriksa dan memperbaharui dengan sesuai permission pada Task Execution Role agar dapat mengambil secret yang diperlukan dan memastikan lagi ARN secret pada ECS Task definition itu sudah benar
- Kemudian jika sudah melakukan perbaikan, jalankan ulang Task ECS tersebut

Scenario C : Unauthorized AWS API activity

CloudTrail shows unexpected API calls from an IAM principal used by the deployment pipeline. Describe your immediate containment actions, investigation process, and recovery steps. Consider credential revocation, affected resources, audit evidence, and the possibility that credentials or permissions have been compromised.

Jawaban :

Berarti si Cloudtrail mendeteksi adanya akun IAM atau role yang digunakan pada pipeline itu melakukan aktivitas mencurigakan/tidak sah ataupun tidak sesuai dengan tugas seharusnya

- Pertama, saya akan mengecek kembali akun IAM atau role tersebut dan jika tidak sesuai akan menonaktifkannya, kemudian koordinasi untuk mengehentikan sementara proses pipeline deployment agar tidak berdampak ke aplikasi kita yg berjalan
- Kedua, saya akan periksa AWS Cloudtrail untuk melihat API calls yang mencurigakan, beserta waktunya, IP addrees, dan resource yang telah diakses , kemudian menganalisis dan mengindentifikasi apakah ada resource kita itu yang diubah, dibuat, dan dihapus
- Ketiga, Membuat ulang/ganti credential yang sudah terdampak setelah memastikan bahwa akses mencurigakan itu sudah berhenti, dan memperbaiki konfigurasi serta resource dengan koordinasi dengan tim jika misalkan sudah ada resource yang sudah diubah bahkan dihapus

f. Task 6 : DAST and OWASP Top 10

Demonstrate how you would incorporate DAST into the pipeline using OWASP ZAP or an equivalent tool.

Jawaban :

DAST (Dynamic Application Security Testing), digunakan untuk menguji keamanan dari aplikasi kita yang sedang berjalan. Jadi saya akan mengintegrasikan tool security ke dalam pipeline Gitlab CI/CD untuk melalukan pengujian keamanan aplikasi secara otomatis sebelum aplikasi di rilis ke production

1. Aplikasi akan dijalankan terlebih dahulu di lingkungan pengujian seperti environment testing/staging
2. OWASP ZAP dapat untuk checks otomatis untuk mendeteksi adanya kerentanan seperti: SQL Injection, Cross Site Scripting(XSS)
3. Untuk autentikasi, jika si aplikasi kita membutuhkan credential buat login, kita bisa menggunakan credential khusus buat testingnya
4. Jika ada security findings seperti CRITICAL dan HIGH : maka pipeline kita itu akan gagal dan release akan distop sampai masalah itu fix sudah diperbaiki dan jika ada security findings seperti MEDIUM dan LOW : maka akan tetap tercatat untuk laporan dokumentasi untuk diperbaiki berdasarkan tingkat resikonya
5. Kita dapat mencegah dampak ke environment production dengan kita menjalankan scan hanya pada environment testing/staging yang sudah diizinkan dan memastikan agar tidak mengarah ke environment production

Contoh Resiko OWASP ZAP :

1. SQL Injection, dengan menguji input ke aplikasi untuk menemukan adanya indikasi manipulasi query database, solusinya dilakukan menggunakan parameterized queries dan validasi input

2. Broken Acces Control, dengan menguji apakah user biasa itu dapat mengakses data atau endpoint, solusinya dilakukan dengan menerapkan pemeriksaan(authorization) dari setiap user dan endpoint

3. Security missconfiguration, dengan memeriksa apakah konfigurasi dari server dan security headers kita itu ada yang kurang tepat, solusinya dengan memperbaiki serta memperbaharui konfigurasi yang tepat dan aman.

Note : Dalam pengerjaan test case ini, jujur saya menggunakan bantuan dari AI sebagai referensi serta dokumentasi resmi dari Gitlab CI/CD, OWASP ZAP, TRIVY, TERRAFORM, dan AWS sebagai proses pembelajaran mandiri saya untuk lebih belajar serta improve mengenai konsep dari security untuk posisi DevSecOps ini, Saya juga menyadari bahwa masih banyak hal-hal yang perlu saya pelajari lebih dalam terutama hands on di production, untuk itu saya sangat terbuka untuk menerima masukan untuk proses pengembangan dan pemahaman saya terkait posisi DevSecOps ini.